Solution: Decode String

Problem Statement

You have a string that represents encodings of substrings, where each encoding is of the form `k[encoded_string]`, where `k` is a positive integer, and `encoded_string` is a string that contains letters only.

Your task is to decode this string by repeating the `encoded_string` `k` times and return it. It is given that `k` is always a positive integer.

Examples

1. - **Input**: `"3[a3[c]]"`
   - **Expected Output**: `"acccacccaccc"`
   - **Justification**: The inner `3[c]` is decoded as `ccc`, and then `a` is appended to the front, forming `accc`. This is then repeated 3 times to form `acccacccaccc`.
2. - **Input**: `"2[b3[d]]"`
   - **Expected Output**: `"bdddbddd"`
   - **Justification**: The inner `3[d]` is decoded as `ddd`, and then `b` is appended to the front, forming `bddd`. This is then repeated 2 times to form `bddd bddd`.
3. - **Input**: `"4[z]"`
   - **Expected Output**: `"zzzz"`
   - **Justification**: The `4[z]` is decoded as `z` repeated 4 times, forming `zzzz`.

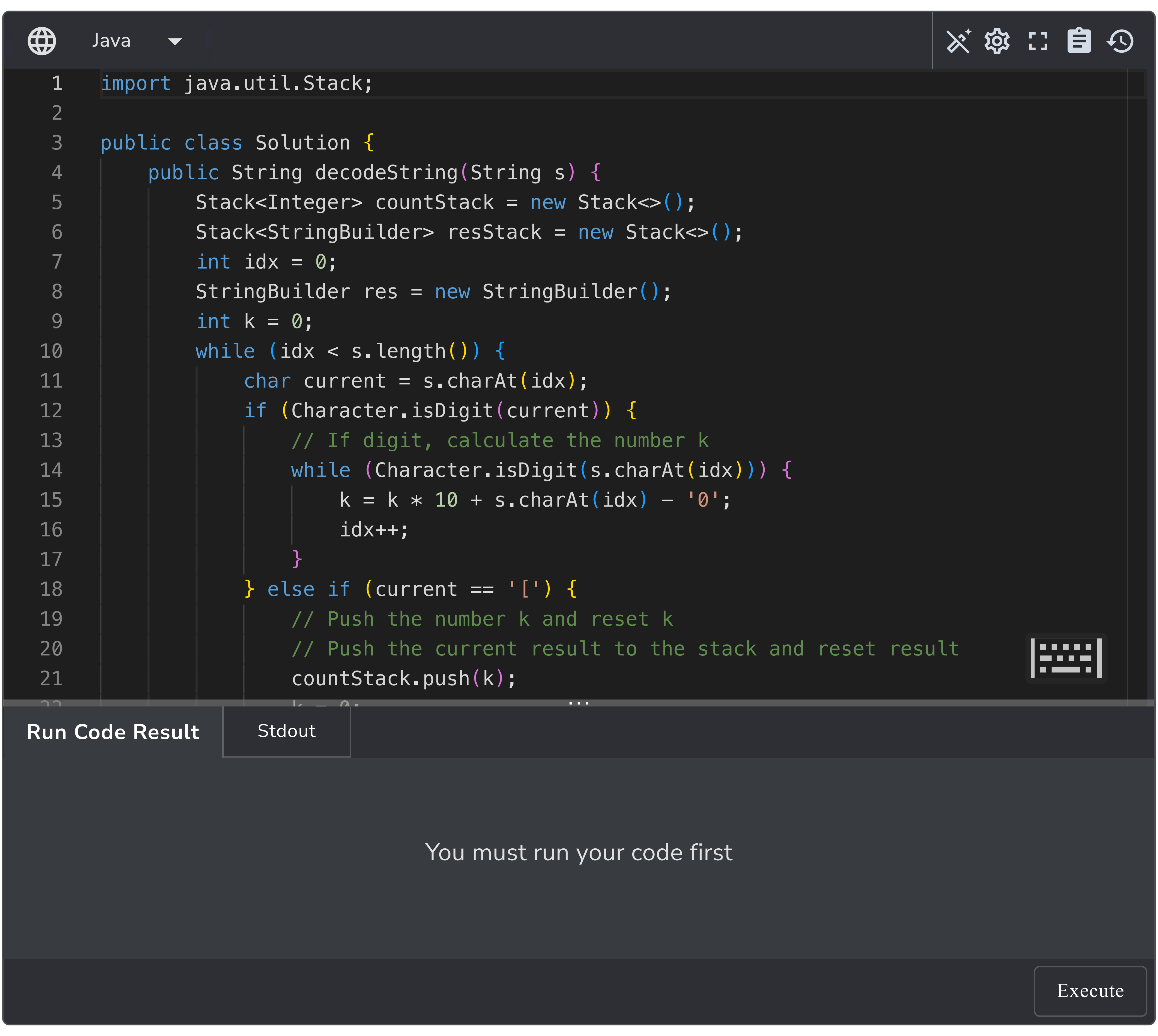
Solution

- **Understanding the Problem**:
  - The problem involves decoding a string that contains patterns where a number is followed by a string in brackets.
  - The number indicates how many times the string in brackets should be repeated.
- **Approach**:
  - Use a stack to keep track of the characters in the string.
  - Iterate through the string character by character.
  - When a number is encountered, calculate the complete number.
  - When an opening bracket `[` is encountered, push the calculated number to the stack.
  - When a closing bracket `]` is encountered, pop elements from the stack until a number is encountered and form the substring to be repeated.
  - Multiply the substring with the number and push the result back to the stack.
- **Handling Nested Brackets**:
  - The stack will naturally handle nested brackets as it will continue popping elements until a number is encountered, forming the substring for the innermost bracket first.

Algorithm Walkthrough

- **Given Input**: `"3[a3[c]]"`
- **Steps**:
  - Initialize an empty stack.
  - Iterate through the string:
    - Encounter `3`, push `3` to the stack.
    - Encounter `[`, do nothing.
    - Encounter `a`, push `a` to the stack.
    - Encounter `3`, push `3` to the stack.
    - Encounter `[`, do nothing.
    - Encounter `c`, push `c` to the stack.
    - Encounter `]`, pop `c` and `3`, form `ccc` and push it back to the stack.
    - Encounter `]`, pop `ccc`, `a`, and `3`, form `acccacccaccc` and push it back to the stack.
  - The final stack contains `acccacccaccc` as the only element, which is the decoded string.

Code



```java
import java.util.Stack;

public class Solution {
    public String decodeString(String s) {
        Stack<Integer> countStack = new Stack<>();
        Stack<StringBuilder> resStack = new Stack<>();
        int idx = 0;
        StringBuilder res = new StringBuilder();
        int k = 0;
        while (idx < s.length()) {
            char current = s.charAt(idx);
            if (Character.isDigit(current)) {
                // If digit, calculate the number k
                while (Character.isDigit(s.charAt(idx))) {
                    k = k * 10 + s.charAt(idx) - '0';
                    idx++;
                }
            } else if (current == '[') {
                // Push the number k and reset k
                // Push the current result to the stack and reset result
                countStack.push(k);
```

You must run your code first

Complexity Analysis

- **Time Complexity**: O(n), where n is the length of the input string. This is because we are iterating through the string once and processing each character.
- **Space Complexity**: O(n), where n is the length of the input string. In the worst case, the stack will store all the characters of the input string.

← Previous
Next →
Decode String (Medium)
Valid Sudoku (Medium)
☐ Mark as Completed